

Using a Wisdom of Crowds Approach on the Bin Packing Problem

Andy Phan, Author, Speed School of Engineering, Undergraduate of the Department of Computer Science, University of Louisville

Abstract—Creation and testing of a Genetic Algorithm modified with the philosophy of the Wisdom of the Crowd, on the NP-Complete problem describing a list of items to be “thrown away” in the Bin Packing Problem (BPP). Using a variety of differently sized and organized items, testing how the algorithm holds up against the Greedy Algorithm in its proposed solutions. A model of solving using a system which collects the most agreed upon solution as per the Wisdom of Crowds philosophy. This paper aims to compare my proposed solutions generated with a Wisdom of Crowds based algorithm, Genetic Algorithm and Greedy Algorithm with each other based on performance and efficiency.

Index Terms—Wisdom of Crowds, Bin Packing Problem, Greedy Algorithm, Genetic Algorithm

I. INTRODUCTION

THE Bin Packing Problem is a classical NP-Hard problem that was introduced in the 1970s as an optimization problem involving several objects and containers.¹ It is a further extrapolation of the Knapsack Problem, where instead of having to fit valuable objects into one container, you are tasked to fit every object into few containers.^{(5) (6)}

The time complexity of the problem is NP-Hard due to how abstract the problem presented could be. The idea is to fit all objects in a list into the smallest number of containers possible, with objects of all different sizes and shapes. For instance, the objects could be two dimensional, three dimensional or even abstract. In one case, the problem could be to fit rectangular objects into a rectangular bin in two dimensions, and in another case, the problem could be to fit spherical objects into a rectangular bin.² For an abstract case, imagine fitting chunks of memory into an SSD, or managing RAM usage.

The problem has real world usage in many different forms other than as a mathematical brain teaser. Questions involving the problem could range from: “How can I fit as many cargo containers into this transport as possible?”² or “What is the best way to pack these containers into this warehouse so that it uses up as much space as possible?” In the computer science field, you can ask: “How will I schedule these CPU tasks so

that there are few CPUs being used?” or “How can I store these files so that the fewest number of storage blocks are used?”²

There are many different solutions presented for solving these problems, however this paper will describe the use of GA, WoC and a Greedy Algorithm to solve a problem of malleable objects fitting in non-descript shaped bins, making a one-dimensional BPP.

II. BIN PACKING PROBLEM AS DESCRIBED IN ONE DIMENSION

A. Description of the One-Dimensional Bin Packing Problem

The one-dimensional BPP is only restricted by one variable. Rather than two dimensions, like the width and length of the objects and the bins, the one-dimensional problem is limited only by the “size” of the bins, with no regard to the shape. For instance, an object of size n may fit into a bin of size k , where the remaining space inside the bin is now defined by $k - n$, as long as $n \leq k$.

The problem states that every object must be stored in some bin, and so when an object no longer fits into any bins, a new bin must be used to have enough space to store it. The most optimal solution requires the solution to have as few bins as possible, with the total space created by all the bins being close to the total space taken up by every individual object. In a more mathematically theoretical sense, there exists a solution in which there are the fewest bins used to store a set S of items.

The calculation of bins is decided by the order in which objects are fit into bins, by object ID. For instance, the genetic code for the order of objects may be $[2, 4, 7, 5, 3, 1, 6]$, in which case object 2 is added to bins first, then object 4 etc. The bins may look like $\{[2], [4, 7], [5, 3, 1], [6]\}$, with each bin containing objects (in this case, 4 bins are used to store 7 objects).

B. Presented Solutions

In this paper, there is an exploration of three different methods to solve the BPP in as few solutions as possible. First is the Greedy Algorithm. This is the most straight-forward algorithm in which the program finds the largest item that fits into the remaining portion of the bin. For instance, objects of sizes $[0.6, 0.8, 0.7]$ may try to fit into the remaining bin space of 0.75, so the object of size 0.7 is chosen as it is the largest object that may fit into the remaining space of the bin. This

Manuscript received November 27, 2025.

A. P. Author is and undergraduate with the University of Louisville, Speed School of Engineering, Louisville, KY 40214 USA (phone: 502-296-8043; e-mail: ahphan02@louisville.edu).

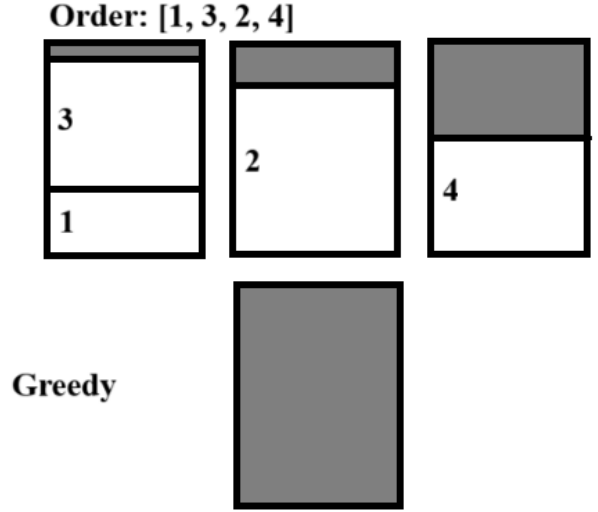
algorithm is also used as a mutation for the GA and WoC interpretations of the problem.

In the GA, the problem is solved by using a series of produced generations, where the previous generation will carry on its best solutions to the next, through crossovers and mutations. For each Individual of a generation, two Parents are chosen from the top 50% of the solutions in the generation to select from their genomes (order in which objects are placed into bins). There is an 80% chance which one of two parents' genes are taken from, and there is a 20% chance that a mutation occurs. From those mutations, there is a 5% chance that a Greedy Algorithm is used to find the next gene, and a 15% chance that a random gene is taken from the remaining gene pool. If neither parent has a gene in the remaining gene pool, then one of the two mutations is chosen at a 50% chance each. A population is made up of 100 individuals, of which only 10 will be chosen to automatically be in the next generation, and the last 90 will be produced by the top 50% of the current generation. The population is sorted by their fitness score, which is the total number of bins that were used, from fewest to most. This will continue for as many generations as necessary (in this case, 100 generations is used) and the most optimal solution would be the Individual with the lowest number of bins used in the last generation.

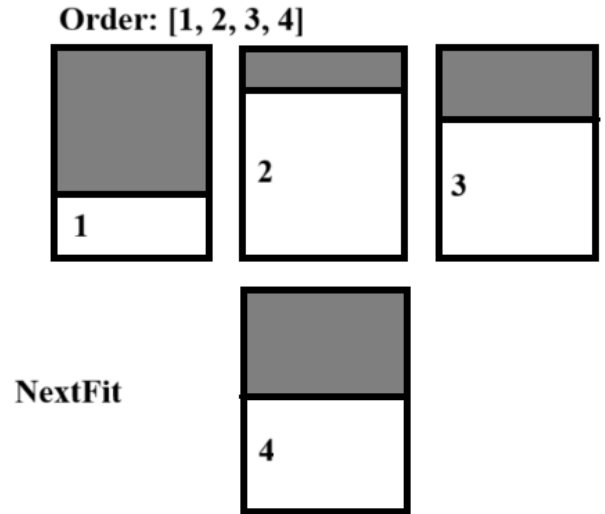
The WoC algorithm works namely the same as the GA, however it is one step extrapolated. Every generation, the top 30% of the generation will have their genes placed into a pool, where the most agreed upon gene will be placed into the solution of that generation. If the most agreed upon gene is not in the pool of unused genes, then they may use the Greedy Algorithm to find the next gene.

A number of different solutions have been presented to solve this problem as well, such as the NextFit algorithm.³ My interpretation of a Greedy Algorithm takes the next step beyond this one, however, the NextFit algorithm will only look at the next item, and decide either to place it into the most recent bin, or to create a new bin to place it into. This is an algorithm that is very uncommonly the most optimal solution, or close to, as it will only run through the list of objects from start to finish. However, the NextFit algorithm does have its own uses. This algorithm is defined by finding a solution that is less than twice the fewest number of bins that is most optimal.³ Therefore, a solution that gets closer to half the number of bins found by the NextFit algorithm would be closer to the optimal solution.

C. Figures of Greedy and NextFit



(a) Greedy Algorithm example



(b) NextFit Algorithm example

As seen in figure (a), the Greedy Algorithm uses one less bin than the NextFit Algorithm in figure (b). This is due to the order in which the objects are placed into bins, as object 1 and object 3 can both fit into one bin, while object 2 needs its own bin. Therefore, the order to place items is different for both algorithms.

III. SOLVING THE ONE-DIMENSIONAL BPP USING WoC, GA AND GREEDY ALGORITHMS

A. Overview of the Wisdom of the Crowd Philosophy

The Wisdom of the Crowd philosophy has been discussed throughout history, and the first person to speak about it on paper is Aristotle.⁷ The Wisdom of Crowds philosophy states that the collective guess of the crowd is much better than the guess of the individual. For example, 800 people were tasked to guess the weight of an ox in 1906. The median guess from

the entire crowd was within 1% of the weight of the ox, which is likely better than most other guesses.

The Wisdom of Crowds philosophy is used in many aspects of society, and the idea of democracy is directly related to it.⁷ We utilize this philosophy to examine its effect on the Genetic Algorithm and the Bin Packing Problem.

B. Solving the One-Dimensional BPP Using WoC, GA and Greedy Algorithms

1.) Supplied Data to Program

The numbers used for the program are as follows. The bin object uses a maximum size of 3, and objects fall under a range from 0.1 to 3.0. The reason why the objects are capped at 3.0 is because an object of size 3.1 may never fit into a bin, causing the problem to be invalid. The size of the population when described is made up of 100 individuals, where each individual stores its own genetic information (order in which objects are placed into bins) and its own fitness (number of bins used, lower is better).

The data used to test the algorithms fall into two categories. One category is the “normal” category, where a number of objects are created with a range of [0.1, 3.0], with an even spread across the board. The other category is the “weird” category, in which a number of objects are created with the ranges [1.0, 1.4] and [2.0, 2.4], to create an environment where objects do not fit neatly into bins (for instance, an object with size 2.4 needs its own bin). The data is stored in the format: “{itemId} {size}”.

There are 8 data sets, 4 normal and 4 weird. The number of objects in each data set were the same for normal and weird, being: 25, 75, 150 and 250.

2.) Optimization Results

The found optimization for each data set followed the same pattern across the board. Neither GA nor WoC algorithms were able to find a better optimization for the data sets than the Greedy Algorithm, however the WoC algorithm always performed better than the GA.

In the tables, the “Utilization%” is calculated by:

$$Utilization\% = \frac{totalsizeofobjects}{maxsizeofbin \times totalnumberofbins} \times 100$$

This number defines the amount of space that is used across all bins, where the higher the percentage means that less empty space exists in the bins. This is important, as a higher utilization percentage means that fewer bins are being used, as the total number of bins is the only variable in this equation that can be changed through optimization.

3.) Tables and Figures for Optimization

Average Utilization% found:

Items Set	Greedy	GA	WoC
250w	76.9800	75.6216 (Generation 63)	76.0690 (Generation 97)
250	95.8272	85.1096 (Generation 96)	89.2184 (Generation 6)
150w	76.45	74.9510 (Generation 4)	76.45 (Generation 4)
150	94.7176	85.6489 (Generation 5)	88.4725 (Generation 51)
75w	75.9933	75.9933 (Generation 36)	75.9933 (Generation 94)
75	97.9630	88.1667 (Generation 3)	92.8070 (Generation 90)
25w	75.9804	75.9804 (Generation 1)	75.9804 (Generation 3)
25	91.4103	91.4103 (Generation 8)	91.4103 (Generation 75)

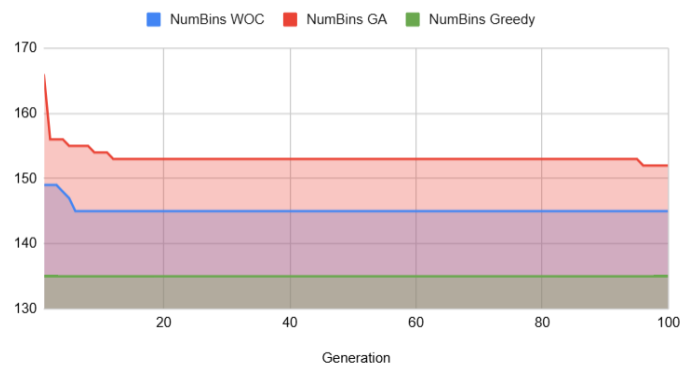
(c) Utilization% for all three algorithms along with the generation it produced it with (if viable). The “w” tag after numbers in the “Items Set” column denotes whether the data set was weird.

Number of Bins found:

Items Set	Greedy	GA	WoC
250w	167	170	169
250	135	152	145
150w	100	102	100
150	85	94	91
75w	50	50	50
75	36	40	38
25w	17	17	17
25	13	13	13

(d) Number of Bins found for all three algorithms, similarly to figure (c). The “w” tag after numbers in the “Items Set” column denotes whether the data set was weird.

NumBins vs. Generation (250)



(e) Graph displaying the generational improvement of the algorithm finding progressively fewer bins for GA and WoC on the data set for 250 items.

4.) Elaboration on Optimization Results

For data sets featuring fewer objects, the difference between the three data sets is not apparent based on the optimized solution. For the data sets involving 25 items, weird or normal, all three solutions contained the same number of bins, see Figure (d). The weird data sets up to 250 items did not provide a significant enough difference between the three data sets to warrant further elaboration, besides the fact that every weird data set had an optimal solution with more bins than their normal counterparts.

For each of the normal data sets, the difference is much more apparent between the GA and WoC solutions as the number of objects to be stored increases.

GA vs. WoC in terms of Optimization:

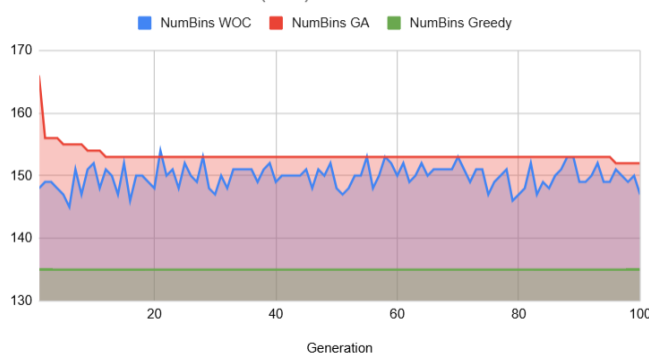
Items Test	GA	WoC	+%Optimization
250w	170	169	+0.59
250	152	145	+4.61
150w	102	100	+1.96
150	94	91	+3.19
75w	50	50	-
75	40	38	+5.00
25w	17	17	-
25	13	13	-

(f) The table depicts the GA and the WoC solutions for each of the data sets, along with the percent increase in optimization.

From the table in Figure (f), the optimization percentage is more apparent for normal data sets, ranging from 3-5% improvement from 75 objects and above.

An interesting thing to note about the WoC algorithm, is that the optimal solution per generation varied throughout the process of running.

NumBins vs. Generation (250)



(g) The graph depicts the best solution found for each generation of the WoC and GA applications.

As seen in Figure (g), the agreed upon solution varied throughout the lifetime of the process, however, for 250 objects, the agreed upon solution was almost always better than the best solution found for the GA implementation.

The solutions for GA and WoC fail to find a better solution than the Greedy Algorithm in most cases, varying from 5-10% less optimized on data sets larger than 75.

From these graphs and tables however, we can determine that WoC is an applicable philosophy to improve the proposed solutions.

5.) Efficiency Results

Each program was also tested on its efficiency, or how long it took each program to run for 100 generations (barring the Greedy Algorithm, as it only needs to run once). The measurements for efficiency is provided in seconds for each number of the data sets, as the difference between a weird and normal dataset was insignificant when comparing efficiency.

6.) Tables and Figures for Efficiency

Time Taken(seconds) to run each test:

Items Test	Greedy	GA	WoC
250	2.30	158.0	217.74
150	1.15	85.7	126.84
75	0.29	22.97	25.54
25	0.04	5.17	4.99

(h) The table depicts the time taken for each number of objects tested for Greedy, GA and WoC algorithms.

item count vs. run time



(i) The graph depicts the table (h).

7.) Elaboration on Efficiency Results

The WoC algorithm takes the longest to run, then the GA and then the Greedy. A proposed reason for this, is that for WoC, each generation needs to find the most agreed upon solution, which means that 30% of the generation has further algorithms to run through compared to the GA.

For WoC and GA, the time taken for the number of objects was exponentially increasing, where 25 items takes ~5 seconds, 250 items takes multiple minutes. The WoC solution is ~30% slower than the GA solution, and the Greedy Algorithm beats both of them by a long shot.

IV. CONCLUSION

In conclusion, the application of the Wisdom of the Crowd philosophy is better at finding a proposed solution than the Genetic Algorithm on the One-Dimensional Bin Packing Problem. Indeed, in most cases, the average guess of the top 30% of the generation is better than the best solution presented by the Individual with the lowest fitness of a generation. When developing a program for the Bin Packing Problem, it is more

optimal to utilize the Wisdom of Crowds philosophy.

REFERENCES

- [1] L. Andrea, "Bin Packing Problem," in *ScienceDirect* sciencedirect.com, Accessed on 25 Nov 2025 [Bin Packing Problem - an overview | ScienceDirect Topics](#).
- [2] W. Nan, "Two-dimensional Bin-packing Problem with Rectangular and Circular Regions Solved by Genetic Algorithm" *International Journal of Applied Mathematics* 2021, pp. 268-278.
- [3] F. Hiroshi, "Algorithm NextFit for the Bin Packing Problem". *Formalized Mathematics* issue 3, vol. 29, Dec 2021, pp. 141-151
- [4] L. Bingchen, "Identify Patterns in Online Bin Packing Problem: An Adaptive Pattern-Based Algorithm" *symmetry* published 23 June 2022.
- [5] M. Silvano, "Bin Packing Problem" *Knapsack Problems: Algorithms and Computer Implementations* 1990 pp. 221-245.
- [6] "Bin Packing Problem," *Wikipedia*., Available at: [Bin packing problem - Wikipedia](#) (accessed 25 Nov 2025).
- [7] "Wisdom of the Crowd," *Wikipedia*., Available at: [Wisdom of the crowd - Wikipedia](#) (accessed 25 Nov 2025).

Andy P. Author This author is an undergraduate student studying computer science at the Speed School of Engineering at the University of Louisville, under the Computer Science Department. The University resides in Louisville, KY, 40214 USA, along with the student.